

VORBEREITUNG ZUR VORLESUNG

Handout 09 – Zusatzthemen: Multiarch, Docker-in-Docker, Swarm

Modul	3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema	Über den Kurs hinaus: Architekturen, verschachtelte Container, Cluster
Dozent	Dr. Ulrich Winkler
Semester	5. Semester · Informationstechnik

Worum geht es hier?

Drei weiterführende Themen, die im Kurs eine Rolle spielen, ohne im Mittelpunkt zu stehen. Sie erklären, *warum* manches so eingerichtet ist (Docker-in-Docker) und *wohin* die Reise geht (Multiarch, Swarm). Zum Lesen und optionalen Ausprobieren – unbewertet.

1. Multi-Architecture-Images (Multiarch)

Das Problem: verschiedene Prozessor-Architekturen

Ihr Rechner nutzt eine bestimmte CPU-Architektur. Verbreitet sind heute:

- **amd64** (auch `x86_64`): klassische Intel-/AMD-Prozessoren, die meisten Server und Cloud-Maschinen (auch GitHub Codespaces).
- **arm64** (auch `aaarch64`): Apple Silicon (M1–M4), viele Raspberry Pis und immer mehr Server – gerade im **IoT**-Umfeld allgegenwärtig.

Ein Image, das nur amd64-Binärdateien enthält, läuft auf einem ARM-Rechner nicht nativ (bestenfalls langsam emuliert). Für einen IoT-Studiengang ist das keine Randnotiz: Ihr Code soll auf dem Entwickler-Mac **und** auf dem ARM-Gerät im Feld laufen.

Die Lösung: ein Name, mehrere Architekturen

Ein **Multiarch-Image** bündelt Varianten für mehrere Architekturen unter *einem* Namen. Docker lädt beim `pull` automatisch die passende Variante. Möglich macht das eine **Manifest-Liste**: ein kleines Verzeichnis, das auf die einzelnen Architektur-Images verweist. Die offiziellen Images (`python`, `postgres`, ...) sind bereits multiarch – deshalb funktionieren die Übungen auf Mac wie in Codespaces.

Selbst bauen mit buildx

```
# Baut ein Image für zwei Architekturen und schiebt es in eine Registry
docker buildx build --platform linux/amd64,linux/arm64 \
  -t meinname/log-tool:1.0 --push .
```

`buildx` ist Dockers erweitertes Build-System; für fremde Architekturen nutzt es Emulation (QEMU). Beim Bauen *für die eigene* Architektur brauchen Sie das nicht.

Merke

„It works on my machine,“ bekommt mit unterschiedlichen CPU-Architekturen eine neue Bedeutung. Multiarch-Images sorgen dafür, dass derselbe Image-Name auf amd64 und arm64 das Richtige tut.

2. Docker-in-Docker (DinD)

Was ist das?

Docker-in-Docker bedeutet: In einem Container läuft ein *eigener* Docker-Daemon, der selbst wieder Container startet. Genau darauf beruht die Übungsumgebung dieses Kurses. Der Dev-Container (bzw. Ihr Codespace) enthält per Feature `docker-in-docker` einen vollständigen Docker – deshalb können Sie darin `docker run`, `docker build` und `docker compose` ausführen, als säßen Sie auf einem normalen Rechner.

DinD vs. „Docker-outside-of-Docker“

Es gibt zwei Wege, Docker in einem Dev-Container verfügbar zu machen:

- **Docker-in-Docker** (unser Weg): ein *eigener*, gekapselter Daemon im Container. Container, Images und Volumes bleiben in dieser Umgebung. Bind Mounts auf den Arbeitsordner funktionieren, weil der innere Daemon dieselben Dateien sieht.
- **Docker-outside-of-Docker**: der Container nutzt den Daemon des *Wirts* (über dessen Socket). Leichtgewichtiger, aber Bind-Mount-Pfade beziehen sich auf den Wirt – was in Codespaces zu Verwirrung führt. Deshalb haben wir DinD gewählt.

Ein Hinweis zu Rechten und Sicherheit

Ein Docker-Daemon im Container braucht erweiterte Rechte (er läuft *privilegiert*). Für eine **Lern- und Wegwerf-Umgebung** wie einen Codespace ist das in Ordnung. In der Produktion vermeidet man privilegierte Container, wo es geht – etwa mit alternativen Build-Werkzeugen, die keinen laufenden Daemon benötigen.

3. Docker Swarm – vom einen Rechner zum Cluster

Was Compose (noch) nicht kann

Ihr Compose-Stack läuft auf **einem** Rechner. Fällt der aus, ist alles weg; und die Leistung ist durch diese eine Maschine begrenzt. **Docker Swarm** verbindet mehrere Docker-Hosts zu einem **Cluster** und betreibt Container darüber verteilt.

Grundbegriffe

- **Node**: ein Docker-Host im Swarm. **Manager-Nodes** steuern den Cluster, **Worker-Nodes** führen die Arbeit aus.
- **Service**: die deklarative Beschreibung „betreibe *n* Kopien dieses Containers“.
- **Task / Replica**: eine einzelne laufende Kopie eines Service. `--replicas 5` heißt: fünf Tasks, über die Nodes verteilt.
- **Desired State Reconciliation**: Der Manager überwacht laufend den Ist-Zustand. Stürzt eine Replica ab, startet er automatisch Ersatz – bis der Soll-Zustand wieder stimmt.
- **Overlay-Netzwerk**: ein virtuelles Netz, das sich über *mehrere* Hosts spannt, damit verteilte Container sich wie im selben Netzwerk erreichen.

Ein Ein-Knoten-Swarm zum Ausprobieren

```
docker swarm init
docker service create --name web --replicas 3 -p 8081:80 nginx
docker service ls          # 3/3 Replicas
docker service scale web=5 # hochskalieren
docker service ps web      # wo laufen die Tasks?
docker swarm leave --force # aufräumen
```

Einordnung: Compose, Swarm, Kubernetes

- **Compose** – mehrere Container auf *einem* Rechner. Ideal für Entwicklung und kleine Deployments (und diesen Kurs).
- **Swarm** – Dockers eingebauter Weg, dasselbe über *mehrere* Rechner zu verteilen. Einfach zu lernen, weil nah an Compose.
- **Kubernetes** – der Industriestandard für große Cluster: mächtiger, aber deutlich komplexer. Wer Swarm verstanden hat, kennt die Grundideen (Services, Replicas, Reconciliation) bereits.

Zum Weiterlesen

Docker Swarm: docs.docker.com/engine/swarm · Multiarch/buildx: docs.docker.com/build/building/multi-platform · Dev-Container-Features: containers.dev